DocuSage AI

Document Intelligence using Retrieval-Augmented Generation (RAG)

Name of the Team members	Roles
B. Harsha Vardhan	Team Lead and System Architect
K. Akhil Krishna	Document Processing Developer
A. Srinu	Embedding and Index Developer
K. Vinay	Retrieval and Citation Developer
G. Vikram	Interface, Testing and Documentation Lead

1. Introduction

1.1 Project Overview

DocuSage AI is a document question-answering system for institutional content such as handbooks, syllabi, circulars and policies. A user uploads PDF, TXT, Excel and PowerPoint files, asks a question in natural language, and receives an answer that is grounded in the uploaded material and carries source references. When the documents do not contain enough evidence, the system replies “not found” instead of guessing.

The system is built in Python with a Streamlit interface. Text is extracted from each file together with its file, page, sheet or slide metadata, split into chunks, converted to embeddings and stored in a FAISS or Chroma vector index. At question time the most relevant chunks are retrieved

and passed to a language model through an LLM API, whose key is supplied through environment variables.

1.2 Problem Statement

Institutional information is spread across many files and formats. Users must open several documents to answer one simple question, and the wording of a question often differs from the wording of the source text. Manual search is slow and keyword search misses related passages. General chatbots can give plausible but unsupported answers, and answers without references are hard to verify.

1.3 Objectives

- Accept PDF, TXT, Excel and PowerPoint files through a simple upload interface.
- Extract text with file, page, sheet or slide metadata so every passage can be traced to its source.
- Split text into chunks and generate embeddings for semantic search.
- Retrieve the semantically relevant chunks for each question, even when the wording differs from the source.
- Generate answers that are grounded in the retrieved evidence and carry source references.
- Reply “not found” when the evidence is insufficient, rather than producing an unsupported answer.
- Keep the design modular and protect secrets: API keys are never hard-coded in source files.

1.4 Scope

In scope: multi-format document upload, text extraction with metadata, chunking, embedding generation, a FAISS or Chroma vector index, top-k

semantic retrieval, grounded answer generation, citations, abstention, an inventory of indexed documents, and an evaluation plan.

Out of scope: OCR for scanned PDFs, role-based access and audit logs. These are listed under future enhancements.

2. Key Features

- **Multi-format upload:** PDF, TXT, Excel and PowerPoint files, with an inventory of the documents that are currently indexed.
- **Metadata-preserving extraction:** text is read with the file name and the page, sheet or slide it came from.
- **Chunking and embeddings:** extracted text is split into chunks, converted to vectors and stored in a FAISS or Chroma index.
- **Semantic retrieval:** the top-k chunks most similar in meaning to the question are retrieved, which handles questions worded differently from the source.
- **Grounded answers:** the language model is given only the retrieved evidence and is asked to answer from it.
- **Citations:** every answer carries file, page, sheet or slide references so the user can verify it.
- **Abstention:** if the retrieved evidence falls below a threshold, the system replies “not found”.
- **Evaluation plan:** test categories and metrics for retrieval relevance, answer correctness, citation precision, abstention quality and latency.
- **Security and privacy:** approved documents only, API keys in environment variables, source metadata kept with every chunk and retrieved passages available for checking.

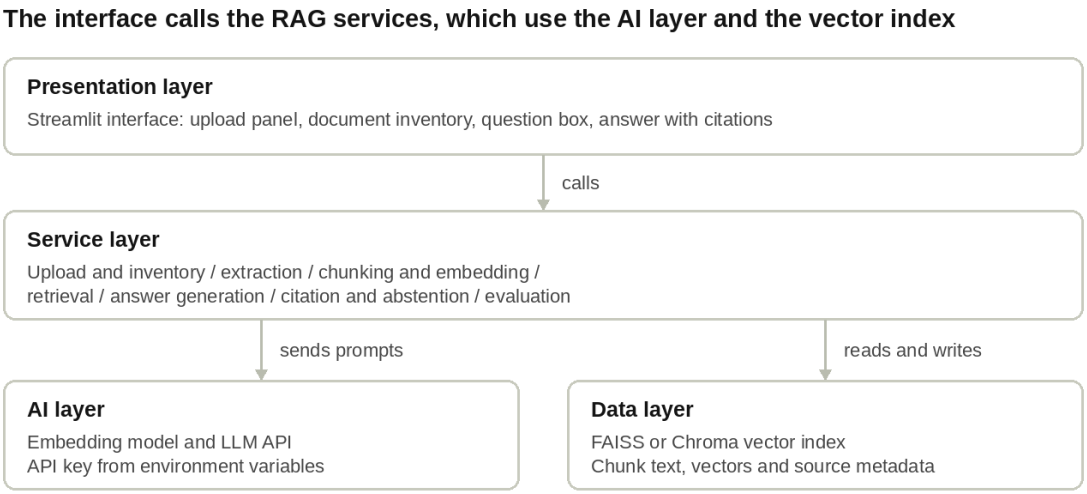
3. Technology Stack

Layer	Technology	Purpose
Language	Python	Application code
Web UI	Streamlit	Upload, inventory, question and answer interface
Application logic	Python modules	Modular parsing, retrieval, prompting and interface logic
Document parsing	PDF, text, spreadsheet and slide parsers	Text extraction with page, sheet and slide metadata
Embeddings	Embedding model via the selected runtime or API	Converting chunks and questions into vectors
Vector index	FAISS or Chroma	Storing vectors and metadata; similarity search
Answer generation	LLM API, configured through environment variables	Grounded answers with citations
Configuration	Environment variables	API keys and settings kept out of source code
Evaluation	Test question set and metrics	Measuring retrieval and answer quality

4. System Architecture

4.1 Layered Design

The application follows a layered architecture. The Streamlit interface contains only presentation logic and calls into the RAG services; the services work with the vector index; the language model and embedding model are reached through the AI layer so that parsing, retrieval, prompting and interface logic stay separate.



layered architecture · 4 layers, 3 calls

Read top down: the interface calls the services, and the services use the AI layer and the data layer.

4.2 End-to-End Workflow

Indexing (documents to vector index):

1. The user uploads PDF, TXT, Excel or PowerPoint files.
2. Text is extracted from each file together with its file, page, sheet or slide metadata.
3. The text is split into chunks, each keeping its source metadata.

4. Embeddings are generated for the chunks and stored in the FAISS or Chroma index.

Answering (question to cited response):

1. The user asks a question in natural language.
2. The question is embedded and the top-k most similar chunks are retrieved.
3. An evidence check compares the retrieval scores with a threshold.
4. If the evidence is sufficient, a grounded prompt is sent to the LLM and the answer is returned with citations; otherwise the system replies “not found”.

4.3 Project Structure

The project is organised into the following logical modules. Folder and file names may differ in the repository.

- **Interface:** the Streamlit application with the upload panel, document inventory, question box and answer view.
- **Upload and Inventory:** accepts files and keeps a list of what is indexed.
- **Extraction Layer:** parsers for PDF, TXT, Excel and PowerPoint files.
- **Chunking and Embedding:** chunk splitting, embedding generation and index building.
- **Vector Retrieval:** top-k similarity search over the index.
- **Answer Generator:** prompt construction and the LLM call.
- **Citation and Abstention:** evidence threshold check and source formatting.
- **Evaluation Module:** test questions and quality metrics.
- **Configuration:** settings and API keys read from environment variables.

5. Functional Modules

5.1 Configuration

Settings are read from environment variables so that no secret appears in source code. The design calls for the following configurable parameters; their default values are to be fixed during implementation.

- **LLM access:** the API key and the model name for answer generation.
- **Embeddings:** the embedding model or runtime to use.
- **Vector index:** the choice between FAISS and Chroma and the storage location.
- **Chunking:** chunk size and overlap.
- **Retrieval:** the number of chunks (k) returned for each question.
- **Evidence threshold:** the minimum retrieval score needed before an answer is attempted.

5.2 Upload and Inventory

- **upload_files:** accepts PDF, TXT, XLSX and PPTX files from the user and passes them to the extraction layer.
- **list_indexed_documents:** shows which documents are currently in the index, so users know what the system can answer from.

5.3 Extraction Layer

extract_text routes each file to the correct parser (PDF, plain text, spreadsheet or slide deck) and returns the text together with its metadata: the file name and the page, sheet or slide number. Keeping this metadata from the first step is what makes citations possible later. Scanned PDFs need OCR, which is not part of the current scope, and tables and slides convert to text imperfectly.

5.4 Chunking and Embedding

- **chunk_text:** splits extracted text into overlapping chunks and attaches the source metadata to each one.
- **generate_embeddings:** converts chunks, and later questions, into vectors using the embedding model.
- **build_index:** stores the vectors and their metadata in the FAISS or Chroma index.

5.5 Vector Retrieval

retrieve_top_k embeds the question and returns the k most similar chunks together with their similarity scores and source metadata. Because the comparison is on meaning rather than exact words, a question can find a relevant passage even when the wording differs. Retrieval can still miss evidence, which is why the next step checks the scores.

5.6 Answer Generation, Citation and Abstention

- **check_evidence:** compares the retrieval scores with the configured threshold and decides whether an answer should be attempted.
- **build_prompt:** combines the question and the retrieved passages into a prompt that tells the model to answer only from the supplied evidence.
- **generate_answer:** calls the LLM API and returns an answer limited to that evidence.
- **format_citations:** attaches the file, page, sheet or slide references of the passages used.

The decision rule can be summarised as follows:

```
if best_score >= threshold: answer with citations, else: reply “not found”
```

RAG improves traceability, but it does not guarantee correctness: retrieval can miss evidence and a model can still misread it. For this reason the retrieved passages are kept available so the user can check the answer.

5.7 Evaluation Module

evaluate_system runs a set of test questions through the pipeline and computes the evaluation metrics described in Section 9: retrieval relevance, answer correctness, citation precision, abstention quality and latency.

6. Security and Privacy

6.1 Document Governance

- Use only approved documents.
- Do not upload confidential files without permission.
- Remember that uploaded content is sent to the embedding and LLM services in the form of chunks and prompts.

6.2 Secrets Management

API keys are supplied through environment variables and are never hard-coded in source files.

6.3 Provider Data Terms

Review the data terms of the LLM provider and of any embedding API before uploading institutional documents.

6.4 Traceability and Risk Mitigation

Risk	Mitigation
Unsupported or invented answers	Grounded prompt and an evidence threshold; the system abstains when evidence is missing
Answers that cannot be verified	Source metadata kept with every chunk and shown as citations
Retrieval misses evidence	Retrieved passages shown for checking; evaluation of retrieval relevance
Confidential files uploaded	Approved documents only; permission required for confidential files
Exposure of API keys	Keys held in environment variables, not in source code
Scanned PDFs and complex tables or slides	Known limitation; OCR is planned as a future enhancement

6.5 Planned Controls

Role-based access and audit logs are not part of the current scope and are listed under future enhancements in Section 12.

7. Data and Index Design

7.1 Overview

DocuSage AI does not need a conventional relational schema. Its data is a set of chunks stored in a FAISS or Chroma vector index. Each chunk holds its text, its embedding vector and the metadata that identifies where it came from.

7.2 Chunk Record

The following is the proposed record layout for each stored chunk.

Field	Purpose	Notes
Chunk text	The passage given to the model as evidence	Produced by chunk_text
Embedding vector	Used for similarity search	Produced by generate_embeddings
File name	Identifies the source document	Used in citations
Page, sheet or slide	Locates the passage inside the file	Used in citations

7.3 Index Lifecycle

- **Add:** uploaded files are extracted, chunked, embedded and added to the index.
- **List:** the inventory shows which documents are indexed.
- **Refresh and removal:** updating or deleting an indexed document is listed as a future enhancement.

7.4 Answer States

- **Answered with citations:** the evidence passed the threshold and a grounded answer was produced.
- **Not found:** the evidence was insufficient and the system declined to answer.

8. User Interface

The interface is built with Streamlit. It keeps presentation logic separate from the RAG services and is designed around the following elements.

- **Upload panel:** select PDF, TXT, Excel or PowerPoint files to add to the index.
- **Document inventory:** the list of documents that are currently indexed.
- **Question box:** a natural-language question field.
- **Answer view:** the grounded answer with its file, page, sheet or slide citations, or a clear “not found” message.
- **Evidence view:** the retrieved passages, shown so the user can check the answer; showing them beside the answer is a planned enhancement.

9. Data, Testing and Quality Assurance

9.1 Test Data

The test set should include approved institutional documents in each supported format (PDF, TXT, Excel and PowerPoint) together with both answerable and unanswerable questions. A repeatable evaluation dataset is listed as a future enhancement.

9.2 Test Categories

- **Answerable questions:** the answer is present in the documents.
- **Unanswerable questions:** the answer is absent and the system should reply “not found”.
- **Citation checks:** each reference points to the passage that supports the answer.
- **Retrieval checks:** the relevant chunk appears among the top-k results.
- **File-format checks:** PDF, TXT, Excel and PowerPoint files are all read correctly.
- **Index-management checks:** uploaded documents appear in the inventory and in search results.

9.3 Evaluation Metrics

- **Retrieval relevance:** whether the retrieved chunks contain the needed evidence.
- **Answer correctness:** whether the answer matches the source.
- **Citation precision:** whether the cited sources actually support the answer.
- **Abstention quality:** whether the system declines exactly when evidence is missing.
- **Latency:** the time taken to return an answer.

This document describes the evaluation plan only; no performance results are claimed. Results are to be recorded when the evaluation is run.

9.4 Validation Approach

The system is to be validated with answerable and unanswerable questions and with manual citation checks, comparing each cited passage with the original file.

10. Installation and Deployment

10.1 Prerequisites

Python, pip, access to an embedding model and an LLM API key.

10.2 Setup Steps

1. Obtain the project (clone the repository or extract the archive) and open a terminal in the project folder.
2. Create and activate a virtual environment: python -m venv venv, then venv\Scripts\activate on Windows or source venv/bin/activate on Linux and macOS.

3. Install the project dependencies, for example with pip install -r requirements.txt if the project provides that file.
4. Set the environment variables listed below, for example in a .env file that is kept out of version control.
5. Start the Streamlit application (for example streamlit run app.py, using the actual name of the main file) and open the address shown in the terminal.

10.3 Environment Variables

The names below are suggested; use the names that the final code reads.

- **LLM_API_KEY:** API key for answer generation.
- **LLM_MODEL:** model name used for answers.
- **EMBEDDING_MODEL:** embedding model or runtime.
- **VECTOR_STORE:** FAISS or Chroma.
- **TOP_K:** number of chunks retrieved per question.
- **EVIDENCE_THRESHOLD:** minimum score for attempting an answer.

10.4 Deployment Notes

- Keep API keys in environment variables and out of source code and version control.
- Index and serve only approved documents.
- Review the data terms of the LLM and embedding providers before use.
- Store the vector index on persistent storage so documents do not need to be re-indexed at every start.

11. User Guide

11.1 User

1. Open the DocuSage AI application in the browser.

2. Upload one or more approved PDF, TXT, Excel or PowerPoint files and wait for indexing to finish.
3. Check the document inventory to confirm that the files are listed.
4. Type a question in natural language.
5. Read the answer and its citations, and open the cited page, sheet or slide to verify it.
6. If the reply is “not found”, rephrase the question or upload the document that should contain the answer.

11.2 Administrator

The person maintaining the system sets the environment variables, makes sure that only approved documents are indexed, keeps the API keys secret and reviews the evaluation results.

12. Limitations and Future Enhancements

12.1 Current Limitations

- Scanned PDFs need OCR, which is not part of the current scope.
- Tables and slides convert imperfectly to text.
- Retrieval can miss evidence, and a model can still misread the evidence it is given.
- Answer quality depends on the quality of the uploaded documents and of the embedding and LLM services.
- Role-based access and audit logs are not available.

12.2 Future Enhancements

- Document filters and categories.
- Index refresh and removal.
- Answers shown beside the retrieved passages.
- A repeatable evaluation dataset.
- OCR for scanned PDFs.

- Role-based access and audit logs.

13. Team and Responsibilities

Member	Role	Focus areas
B. Harsha Vardhan	Team Lead and System Architect	Overall design and module integration; end-to-end RAG workflow; answer generator and prompt design; project coordination and review
K. Akhil Krishna	Document Processing Developer	Upload and inventory module; extraction layer for PDF, TXT, Excel and PowerPoint; page, sheet and slide metadata
A. Srinu	Embedding and Index Developer	Text chunking; embedding generation; building and managing the FAISS or Chroma index
K. Vinay	Retrieval and Citation Developer	Top-k semantic retrieval; evidence threshold and abstention logic; citation formatting
G. Vikram	Interface, Testing and Documentation Lead	Streamlit interface; test cases and evaluation metrics; configuration and secrets handling; project documentation

14. Conclusion

DocuSage AI shows how retrieval-augmented generation can make institutional information easier to find while keeping every answer traceable to its source. Its metadata-preserving extraction, semantic

retrieval and citations let users verify what they read, and its evidence threshold makes the system decline to answer instead of guessing. The modular design leaves clear room for the enhancements listed above, and the next step is to validate the system with answerable and unanswerable questions and manual citation checks.

Appendix A. Function Reference

The functions below summarise the core logic of each module. The names are descriptive and may be adjusted to match the final code.

Module	Function	Purpose
Upload and Inventory	upload_files	Accept PDF, TXT, XLSX and PPTX files
Upload and Inventory	list_indexed_documents	Show which documents are in the index
Extraction Layer	extract_text	Parse each file and return text with page, sheet or slide metadata
Chunking and Embedding	chunk_text	Split text into overlapping chunks that keep source metadata
Chunking and Embedding	generate_embeddings	Convert chunks and questions into vectors
Chunking and Embedding	build_index	Store vectors and metadata in FAISS or Chroma
Vector Retrieval	retrieve_top_k	Return the k most similar chunks for a question

Module	Function	Purpose
Citation and Abstention	check_evidence	Compare scores with the threshold and decide whether to answer
Answer Generator	build_prompt	Combine the question and passages into a grounded prompt
Answer Generator	generate_answer	Call the LLM and return an answer limited to the evidence
Citation and Abstention	format_citations	Attach file, page, sheet or slide references
Evaluation Module	evaluate_system	Run test questions and compute the metrics

Appendix B. Glossary

- **RAG:** retrieval-augmented generation; retrieving relevant passages and supplying them to a language model so that its answer is grounded in them.
- **Embedding:** a numerical vector that represents the meaning of a piece of text.
- **Chunk:** a short passage of a document, stored with its source metadata.
- **Vector index:** a store such as FAISS or Chroma that finds the vectors most similar to a query vector.
- **Top-k retrieval:** returning the k chunks with the highest similarity to the question.
- **Citation:** a file, page, sheet or slide reference attached to an answer.

- **Abstention:** declining to answer, by replying “not found”, when the evidence is insufficient.
- **LLM:** large language model, used here to write the final answer from the retrieved evidence.
- **Grounded answer:** an answer that is limited to the supplied evidence.
- **OCR:** optical character recognition, which reads text from scanned images.